From regression to classification

Lab 02 · Guided experiments and programming exercises

Turn the equations from the lecture into small, readable NumPy programs. Then distinguish learning a probability from making and evaluating a decision.

> **By the end:** perform a gradient update; train a logistic classifier; compute classification metrics; select a threshold using validation data and evaluate it on held-out test data.

Your files

File	Use
`01_Demos_EN.ipynb`	Run the worked examples, inspect the outputs and answer the observation questions.
`02_Exercises_EN.ipynb`	Complete E1–E4; B1–B3 provide further practice.

Open a notebook

Colab: open colab.research.google.com → File → Upload notebook. Import the demonstration notebook and connect to a Python runtime. Save/download your edited work before leaving.

Local Jupyter: use an existing Python environment, or extract the archive and run these commands in a terminal in that folder:

```
python -m pip install -r requirements.txt
python -m notebook
```

Open the notebook in Jupyter. Run cells from top to bottom with **Shift+Enter**. Both notebooks include their own imports and data. No GPU or API key is needed; a local installation can run without network access once packages are installed.

Read the code

`np.array` creates a numerical vector; `*` multiplies element by element; `np.mean` averages; `np.sum(mask)` counts true values. Array indices start at 0.

`TODO` means that a function is unfinished. `OK` means the supplied checks passed. To resolve inconsistent state, restart the kernel/runtime and run all cells in order.

D1 · Follow one linear update

Predict duration y (minutes) from distance x (kilometres). The lecture data are:

x	1	2	3
y	12	19	32

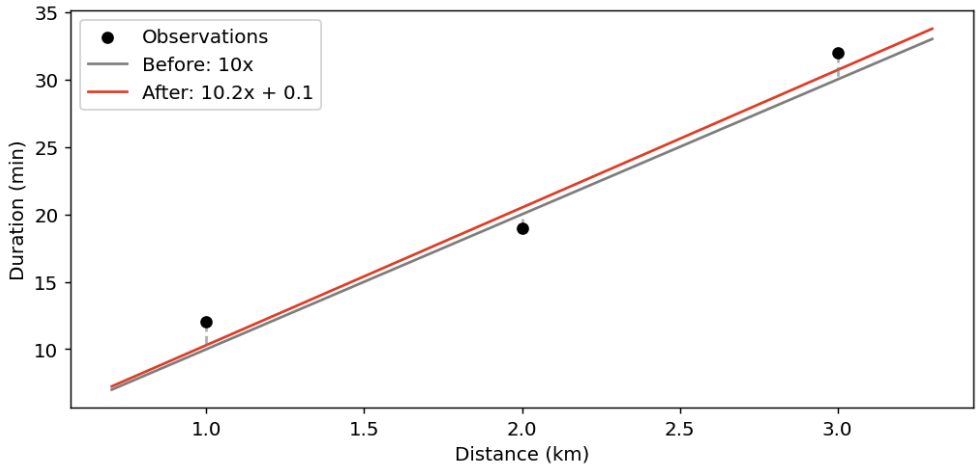
Prediction → Residuals → Mean gradients → Update

$$r_i = wx_i + b - y_i, \qquad J = \frac{1}{2n}\sum_i r_i^2$$

$$g_w = \frac{1}{n}\sum_i r_i x_i, \quad g_b = \frac{1}{n}\sum_i r_i, \quad (w, b) \leftarrow (w - \eta g_w, b - \eta g_b)$$

Start at w=10, b=0 and η=0.1. Read each line before running the D1 code cell.

```
residual = w * x + b - y
dw = np.mean(residual * x)
db = np.mean(residual)
w_new, b_new = w - eta * dw, b - eta * db
```



Expected: r = [−2, 1, −2]; (g_w, g_b) = (−2, −1). The update gives (10.2, 0.1). Loss decreases from 1.5 to 1.138333.

Explain what changed

The dashed segments show the original residuals. A negative gradient means that a small increase of that parameter lowers the loss locally; the update subtracts the gradient.

Question: which objects are data, which are parameters, and which is the learning rate? Why must both gradients use the same old parameters?

D2 · Learn a probability

The target now answers a question: **does the duration exceed 30 minutes?** Define c=1 if y>30, otherwise c=0.

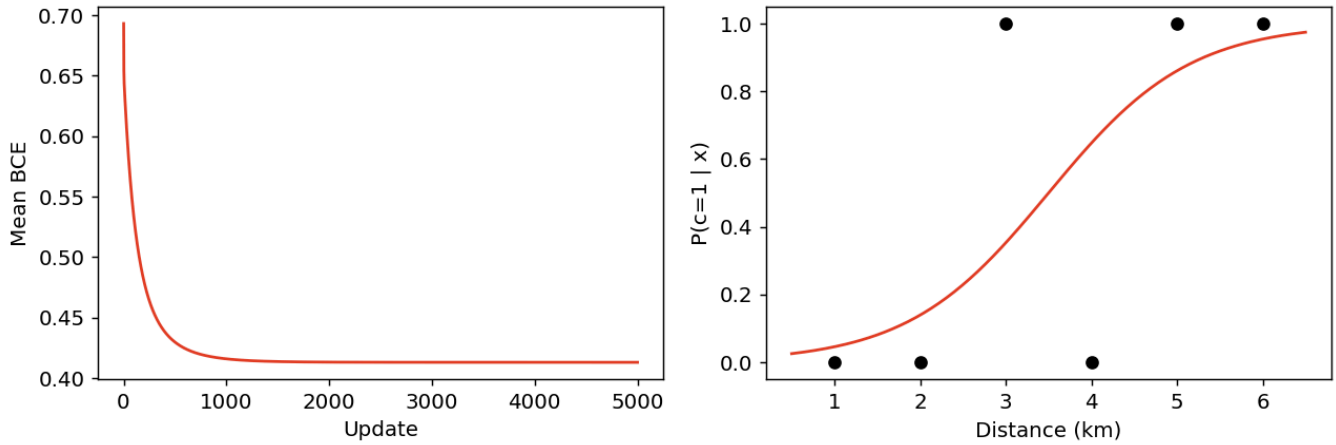
x	1	2	3	4	5	6
y	12	19	32	29	48	57
c	0	0	1	0	1	1

$$p_i = \sigma(wx_i + b), \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

$$J = -\frac{1}{n}\sum_i [c_i \log p_i + (1-c_i)\log(1-p_i)]$$

$$g_w = \frac{1}{n}\sum_i (p_i - c_i)x_i, \qquad g_b = \frac{1}{n}\sum_i (p_i - c_i)$$

1. Run the first-update cell with w=b=0 and η=0.1.
2. Check: every initial probability is 0.5, g_w=−7/12 and g_b=0.
3. Run 5,000 updates and inspect both graphs.



Expected: first loss 0.693147 → 0.665548. After training: w≈1.213446, b≈−4.246823, BCE≈0.412998.

Connect code and theory

`p−c` is the derivative of sigmoid + BCE with respect to the logit. As in D1, average the gradients over the whole batch before updating. All logarithms are natural.

The helper computes the equivalent stable expression `logaddexp(0,z)−c*z`, avoiding numerical overflow in the naive exponential and logarithm formulas.

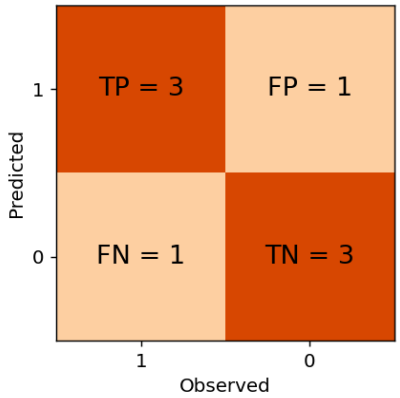
Questions: why are predictions continuous but labels binary? Why does the curve not pass through every label? What remains fixed when the trained model predicts a new input?

D3 · Evaluate a decision

Use the lecture's eight validation scores. They form a separate constructed example, independent of the six training observations in D2.

Case	A	B	C	D	E	F	G	H
p	.95	.85	.70	.60	.45	.30	.20	.10
c	1	1	0	1	1	0	0	0

Predict 1 when **p≥τ**, including equality. Read the matrix at τ=0.5.



TP: predicted positive, truly positive.
FP: false alarm.
FN: missed positive.
TN: correctly rejected negative.

In the lecture, rows are **predicted**, columns are **observed**, with order [1,0]. In scikit-learn, rows are observed: use `confusion_matrix(..., labels=[1,0]).T` to match this layout.

Measure	Calculation	Question answered
Accuracy	(TP+TN)/n	What fraction is correct?
Precision	TP/(TP+FP)	Of positive predictions, how many are true?
Recall / TPR	TP/(TP+FN)	Of true positives, how many are detected?
F1	2TP/(2TP+FP+FN)	Harmonic balance of precision and recall.
Specificity	TN/(TN+FP)	Of negatives, how many are rejected?

Run D3 at τ=0.5 and 0.8. At 0.5, TP/FP/FN/TN=3/1/1/3 and all five measures equal 0.75. At 0.8: 2/0/2/4, precision=1, recall=0.5, F1=2/3.

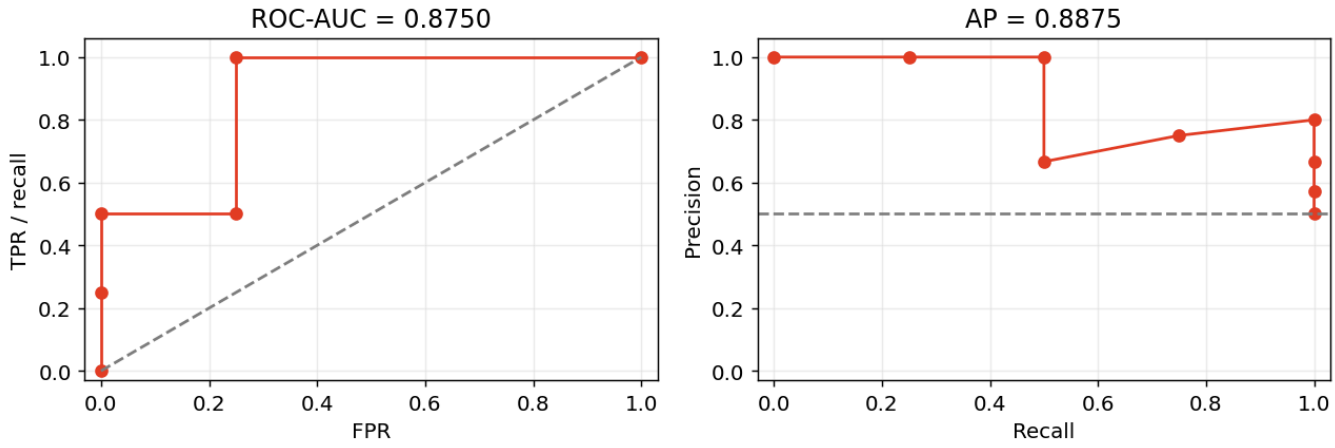
> **Interpret:** accuracy stays 0.75. Which false alarm disappears, and which extra positive is missed? Choose a metric according to the error that matters in the application.

A zero denominator makes the corresponding ratio undefined. This lab reports 0.0 by convention; that convention does not prove the classifier is useful.

D3 · Read ROC and precision–recall

A single threshold produces one decision rule. Varying the threshold produces a curve. Pass the **original scores** to ROC and PR functions.

$$\mathrm{TPR} = \frac{TP}{TP+FN}, \qquad \mathrm{FPR} = \frac{FP}{FP+TN}$$



ROC: ranking across thresholds

ROC plots TPR against FPR. Here AUC=0.875: the positive score wins 14 of the 16 positive–negative pairs. A tied pair contributes half a win.

The diagonal represents random ranking in expectation. AUC measures ranking; it does not say which decision threshold to deploy or whether probabilities are calibrated.

PR: detection and false alarms

PR plots precision against recall. Here the positive prevalence is 4/8=0.5, shown as a reference level. PR is especially informative when positive cases are rare.

$$\mathrm{AP} = \sum_k (R_k - R_{k-1}) P_k$$

In this formula, recall levels increase as the threshold is lowered. For these eight cases, the positive ranks are 1, 2, 4 and 5:

$$\mathrm{AP} = \tfrac{1}{4}\left(1 + 1 + \tfrac{3}{4} + \tfrac{4}{5}\right) = 0.8875$$

Scikit-learn's `average_precision_score` uses recall increments as weights. **Do not replace AP with trapezoidal PR area.** Line segments in the plot connect operating points; they do not define the AP calculation.

> **Questions:** does changing τ from 0.5 to 0.8 change AUC or AP when all scores stay fixed? Can high AUC guarantee high recall at every threshold?

E1–E2 · Implement the learning rules

Open `02_Exercises_EN.ipynb` and run its import cell. Work in the marked functions; keep the supplied checks unchanged.

E1 · A reusable linear update

```
linear_step(x, y, w, b, eta) → (w_new, b_new)
```

Inputs x and y are nonempty 1D NumPy arrays of the same length. Use the D1 loss convention J=MSE/2.

1. Calculate predictions and residuals.
2. Compute both mean gradients from the old parameters.
3. Return the simultaneously updated parameters.
4. Run the checks. The second update starts at (10.2, 0.1), η=0.1 and gives approximately (10.286667, 0.15).

> **Written answer:** explain why updating w before computing the gradient for b changes the algorithm.

E2 · A logistic training function

```
fit_logistic(x, c, eta=0.1, updates=5000)
→ {"w": ..., "b": ..., "losses": ...}
```

Inputs are matching nonempty 1D arrays; c contains 0/1 labels. Initialise w=b=0. Use the supplied `sigmoid` and `mean_bce`.

1. Store the loss before any update.
2. For each update: predict p, compute both mean gradients, update w and b, then append the new loss.
3. Return the final parameters and complete loss history.
4. Run the checks and inspect the loss curve.

Check	Expected
Length of history	`updates + 1`
Zero updates	w=b=0 and one initial loss
Lecture data after 5,000 updates	BCE ≈ 0.412998

> **Written answer:** connect p−c to the derivative of BCE. What does a decreasing training loss establish? What additional evidence is needed for performance on new data?

E3–E4 · Measure and validate

E3 · Your classification report

```
classification_report_manual(y_true, probabilities, tau)
→ {"TP": ..., "FP": ..., "FN": ..., "TN": ...,
   "accuracy": ..., "precision": ..., "recall": ...,
   "f1": ..., "specificity": ...}
```

Inputs are matching nonempty 1D NumPy arrays and binary labels. Predict 1 using $p \geq \tau$. Use boolean masks to count the four outcomes, then the formulas on page 4. Counts must be integers; report **0.0 when a denominator is zero**.

Run the checks for thresholds 0.5 and 0.8, equality at the threshold, and absence of positive cases. Do not call library metric functions inside your implementation.

> **Written answer:** identify the population in precision's denominator and in recall's denominator. Explain how equal accuracy can hide different mistakes.

E4 · A complete evaluation protocol

The supplied synthetic data simulate whether a walk exceeds 30 minutes. A fixed seed produces 200 training, 100 validation and 100 test cases. The code creates the data; the CSV copies are optional.

Train w,b → Select τ on validation → Freeze → Test

```
choose_threshold(y_val, p_val, candidates) → tau
```

1. Run the data cell. Fit w,b with your E2 function on **training only**.
2. In `choose_threshold`, use E3 to evaluate validation F1 at each candidate: [0.2, 0.35, 0.5, 0.65, 0.8].
3. Return the candidate with greatest F1. If candidates tie, choose the **highest threshold**.
4. Inspect the validation table. Freeze (w,b,τ).
5. Run the test cell. Record the confusion counts, the five metrics, ROC–AUC and AP. Do not retune using these results.

> **Written answer:** explain the different roles of training, validation and test. Does the best validation threshold necessarily give the best test F1? Why?

Separate two choices: gradient descent fits probabilities by minimising training BCE. Threshold selection chooses a decision rule by maximising validation F1. Changing τ does not retrain w,b.

Further practice and submission

B1 · Compare learning rates

Reuse E1 on the three linear-regression cases. For each η in [0.01, 0.1, 0.4], start at (w,b)=(10,0) and perform 30 updates. Plot the three losses with a logarithmic vertical axis.

Explain: why keep the data and starting point fixed? Which choice progresses slowly? Which diverges? A falling loss at one learning rate does not guarantee stability at every rate.

B2 · Build ROC from scratch

```
roc_manual(y_true, scores) → (fpr, tpr)
```

Both classes are present. Start above all scores, then lower the threshold through the **distinct scores in descending order**. Group equal scores at the same threshold. Return arrays containing (0,0) and (1,1).

Use the eight D3 cases. Calculate area by trapezoids and recover AUC=0.875. Test one tied positive–negative pair: its AUC must be 0.5.

B3 · Challenge high accuracy

Construct 1,000 labels: 10 positives and 990 negatives. Predict probability 0 for every case, with τ=0.5. Use E3 to report accuracy, recall and F1. Explain whether this classifier can detect positives.

Submit your work

1. Save your completed exercise notebook as `Surname_Name_Lab02.ipynb`.
2. Include E1–E4 code, outputs, written explanations and any extensions attempted. Retain the selected threshold and final test report.
3. Restart the kernel/runtime and run all cells before submitting. Passing checks does not replace explaining your algorithm.

> **Self-check:** can you distinguish a probability from a class, optimisation from generalisation, and a metric at one threshold from a ranking measure?

Read further

- Stanford CS229 · Supervised learning notes: linear and logistic models.
- Cornell CS4780 · Gradient descent and logistic regression.
- scikit-learn · Confusion matrix: row and column convention.
- scikit-learn · Precision–recall: curve and average precision.
- Jupyter · Installation.